

Práctica. CWE-190: Integer Overflow.

Instrucciones. Elaborar una aplicación de consola en C que permita ejemplificar la vulnerabilidad de Desbordamiento de enteros en equipos Linux.

1. Sigue las instrucciones de la práctica en la siguiente página.
2. Crea un reporte de práctica en un **archivo de Word** con una portada con tu nombre. Guárdalo con el nombre **NombreAlumno_CWE-190.docx**.
3. Coloca **las siguientes capturas de pantalla**, cada una con una **descripción textual** arriba de la captura.
 - a. Coloca la captura de pantalla de tu código fuente de C en Visual Studio Code.
 - b. Coloca la captura de pantalla de la salida de tu programa con los ejemplos de desbordamiento de enteros en C.
 - c. Publica tu código fuente en un proyecto público de GitHub. **Coloca la URL** del código fuente publicado en GitHub.
4. Sube tu reporte en un archivo Word a la actividad en Eminus.

Prerrequisitos

1. Esta práctica tiene como prerrequisito haber realizado la práctica de instalación de servidor **Linux**.
Puede seguir las instrucciones de cómo instalar Ubuntu Server desde la práctica [Instalación de Ubuntu server](#).
2. Esta práctica tiene como prerrequisito tener instalado el software **Visual Studio Code** con la extensión para C++.
Puede seguir las instrucciones de cómo instalar Visual Studio Code desde la práctica [Instalación de Visual Studio Code](#).
3. Puede seguir las instrucciones de cómo publicar un proyecto en Visual Studio Code a GitHub desde la práctica [Como publicar un proyecto a GitHub desde Visual Studio Code](#).

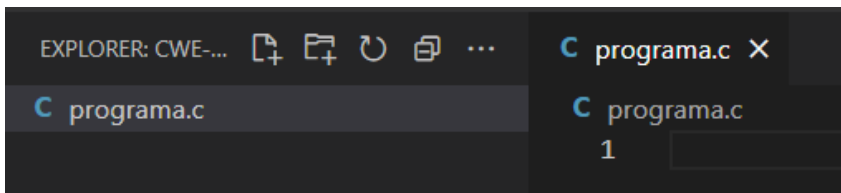
Instrucciones

Un desbordamiento de enteros es una vulnerabilidad en el software donde el cálculo de un valor numérico, pasa del valor máximo que se puede almacenar en una variable entera al valor mínimo que se puede almacenar, en lugar de detenerse en el valor máximo. Esto puede provocar un comportamiento inesperado y potencialmente malicioso, como escribir en ubicaciones de memoria no deseadas o ejecutar código arbitrario.

Vamos a ejemplificar el impacto de una vulnerabilidad de este tipo, mediante una práctica que consista en escribir un programa de código en c que demuestre esta vulnerabilidad en tres escenarios.

A. Crear el programa

1. Asegúrese de contar con una carpeta de trabajo en **C:\codigo**. Si aún no ha creado la carpeta **C:\codigo** hágalo antes de continuar y compártala con su equipo **Linux**. Puede seguir las instrucciones de cómo hacerlo desde [Compartir su carpeta de trabajo de Windows hacia Linux](#).
2. Inicie **Visual Studio Code**.
3. Seleccione **Archivo > Abrir carpeta** en el menú principal.
4. En el cuadro de diálogo **Abrir carpeta**, cree una carpeta en la ruta **C:\codigo\ps\cwe-190** y selecciónela. Luego haga clic en Seleccionar carpeta.
5. El nombre de la carpeta se convierte en el nombre del proyecto y el nombre del espacio de nombres de forma predeterminada. Agregaré código más adelante en el tutorial que asume que el espacio de nombres del proyecto es **cwe-190**.
6. En la barra de herramientas del Explorador de archivos, seleccione el botón **Nuevo archivo**.
7. Asigne un nombre al archivo **programa.c** y VS Code lo abrirá automáticamente en el editor.



8. Escriba el siguiente código.

```
#include <stdio.h>

int main()
{
    int i;
    unsigned int j;

    // Ejemplo 1
    printf("Ejemplo 1\n");
    i = __INT_MAX__; // 2,147,483,647
    printf("i = %d\n", i);
    i++;
    printf("i = %d\n", i);

    j = __UINT32_MAX__; // 4,294,967,295
    printf("j = %u\n", j);
    j++;
    printf("j = %u\n", j);
}
```

```
// Ejemplo 2
printf("\nEjemplo 2\n");
i = -1;
unsigned short u;
u = i;
printf("i = %d\n", i);
printf("u = %hu\n", u);

// Ejemplo 3
printf("\nEjemplo 3\n");
char cresult, c1, c2; // 127
c1 = 100;
c2 = 27;
cresult = c1 + c2;
printf("c = %d\n", cresult);
c1 = 100;
c2 = 28;
cresult = c1 + c2;
printf("c = %d\n", cresult);
}
```

9. Vamos ahora a compilarlo en su equipo **Linux**.
10. Abra una nueva **Terminal**, colóquese en la ubicación de la carpeta compartida de este código. En este ejemplo es `cd codigo/ps/cwe-190/`.
11. Escriba el siguiente comando para compilar su archivo.

```
$ gcc programa.c -o programa.out -fno-stack-protector -z execstack
```

12. Este programa ejemplifica tres casos de desbordamiento de enteros. Ejecute su programa.

```
$ ./programa.out
```

13. El programa esperará que ingrese una cadena. Escriba: `prueba`. Presione **Enter**.

```
pato@servidorfei:~/codigo/ps/cwe-190$ ./programa.out
Ejemplo 1
i = 2147483647
i = -2147483648
j = 4294967295
j = 0

Ejemplo 2
i = -1
u = 65535

Ejemplo 3
c = 127
c = -128
```

14. Observe la salida. En todos los casos, al quedarse sin bits para utilizar, las variables simplemente descartan los bits de la izquierda de sus estructuras e interpretan los bits resultantes como el resultado del número. En realidad realizan la suma correcta, solo que se quedan sin espacio para implementarlo en una computadora.
15. Felicidades. Ha creado su programa correctamente.